

Cryptography Module

Name: Matyas Abraham

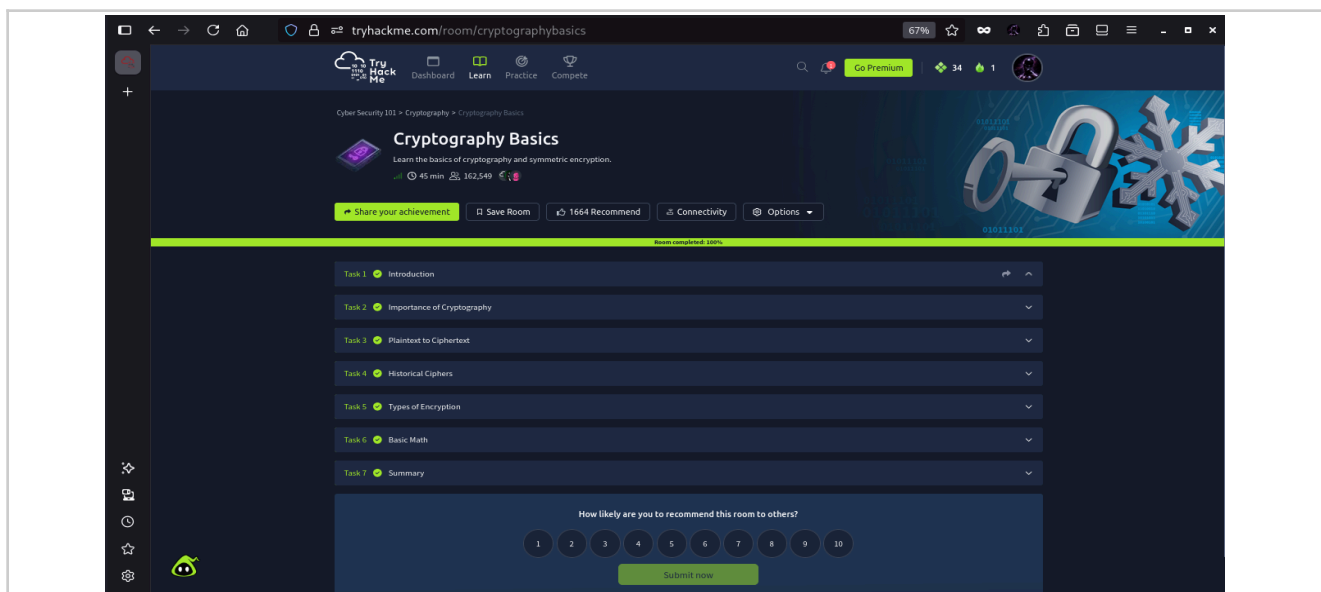
IDNO: CTC-207-26

Overview

The Cryptography module on TryHackMe is made up of four rooms that build on each other: Cryptography Basics, Public Key Cryptography Basics, Hashing Basics, and John the Ripper: The Basics. Together they cover symmetric and asymmetric encryption, how hashing is used to verify data and store passwords, and how password hashes actually get cracked in practice. Room 1 below is covered in full detail with screenshots since it's the one I completed task-by-task; Rooms 2 through 4 are summarized at an overview level.

Room 1 — Cryptography Basics

This room covers the fundamentals of cryptography and symmetric encryption — why encryption matters in the first place, the vocabulary around plaintext and ciphertext, a run through a few historical ciphers, the difference between symmetric and asymmetric encryption, and the small amount of math (modulo and XOR) that keeps showing up underneath it. I went through all seven tasks and finished the room at 100%.



Room overview — all seven tasks completed

Task 1 — Introduction

This task is just the room's setup — it introduces cryptography as the practice of securing information so only the intended recipient can read it, and lays out what the rest of the room will cover: why cryptography matters, the core vocabulary, some historical ciphers, symmetric vs. asymmetric encryption, and the basic math behind it.

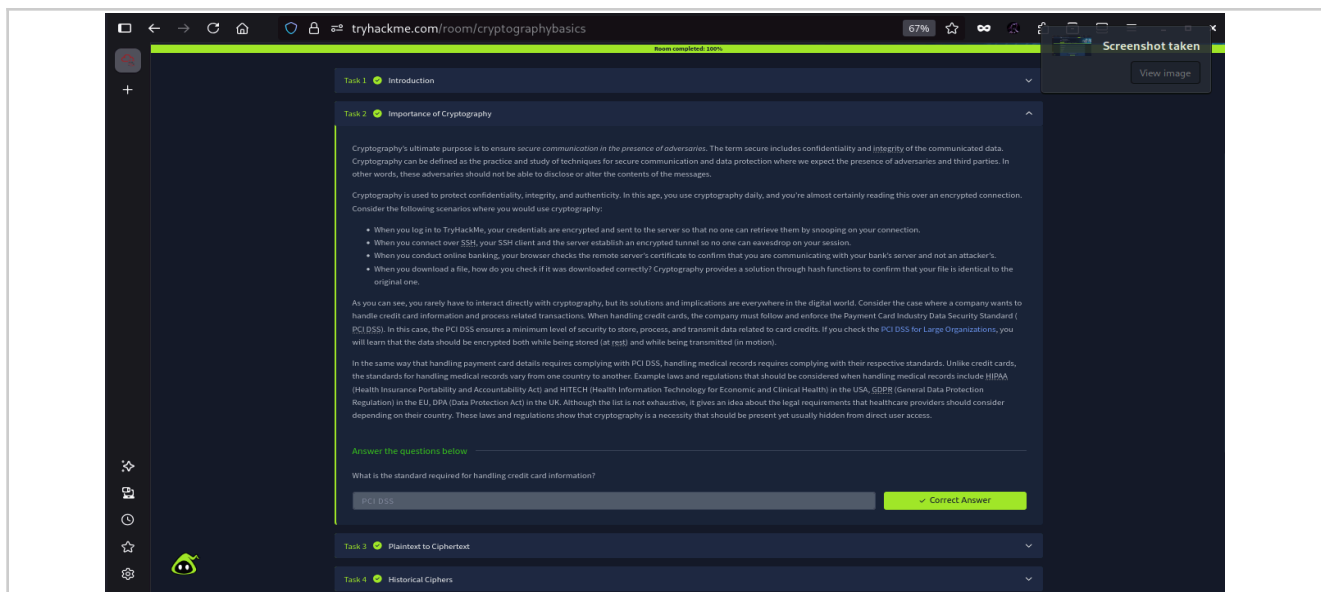
Task 2 — Importance of Cryptography

This task made the case for why cryptography isn't optional in most industries — it's often a legal requirement, not just a good idea. Handling medical records, for instance, is governed by different laws depending on the country: HIPAA and HITECH in the USA, GDPR in the EU, and DPA in the UK. None of these laws mandate encryption by name in every case, but they all expect sensitive data to be protected, and encryption is usually how that gets done in practice.

The task's question was about a different kind of sensitive data — payment information. The standard that applies there is PCI DSS (Payment Card Industry Data Security Standard), which sets the requirements for anyone storing, processing, or transmitting credit card data.

Q: What is the standard required for handling credit card information?

A: PCI DSS.



Task 2 — Importance of Cryptography

Task 3 — Plaintext to Ciphertext

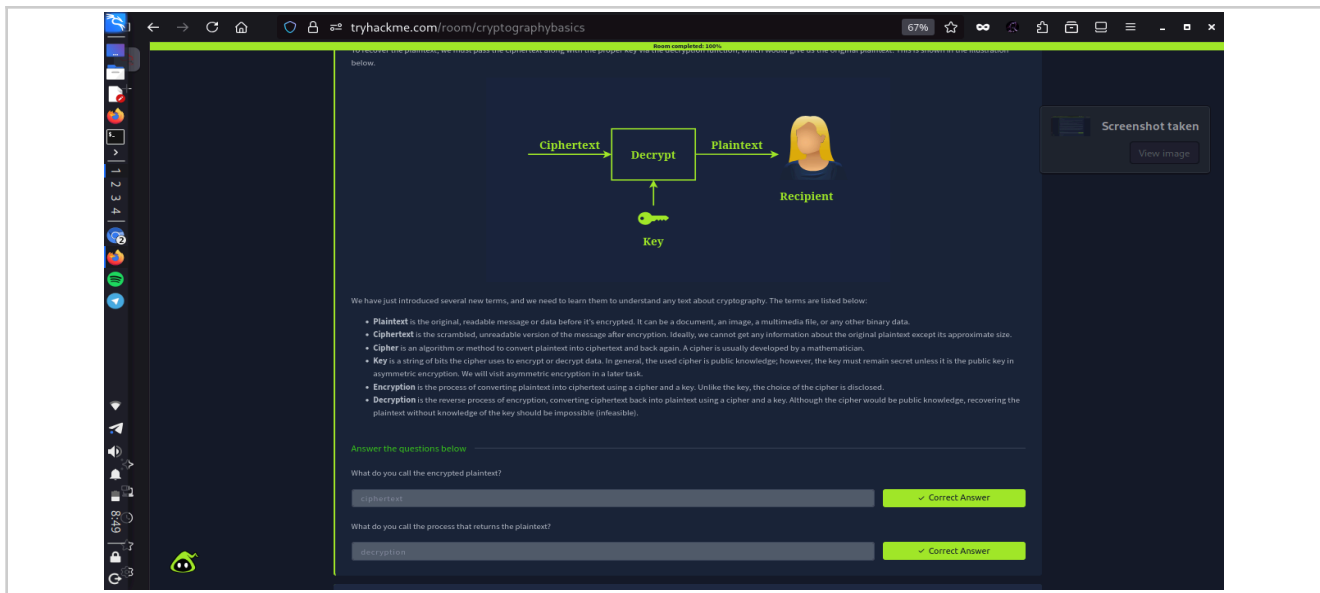
This task covered the core vocabulary: plaintext is the original, readable data before anything is done to it. Encryption is the process of converting that plaintext into ciphertext using a cipher and a key — the cipher can be public knowledge, but without the key, recovering the plaintext should be practically impossible. Decryption is just the reverse: turning ciphertext back into plaintext, again using the cipher and the (correct) key.

Q: What do you call the encrypted plaintext?

A: Ciphertext.

Q: What do you call the process that returns the plaintext?

A: Decryption.



Task 3 — Plaintext to Ciphertext

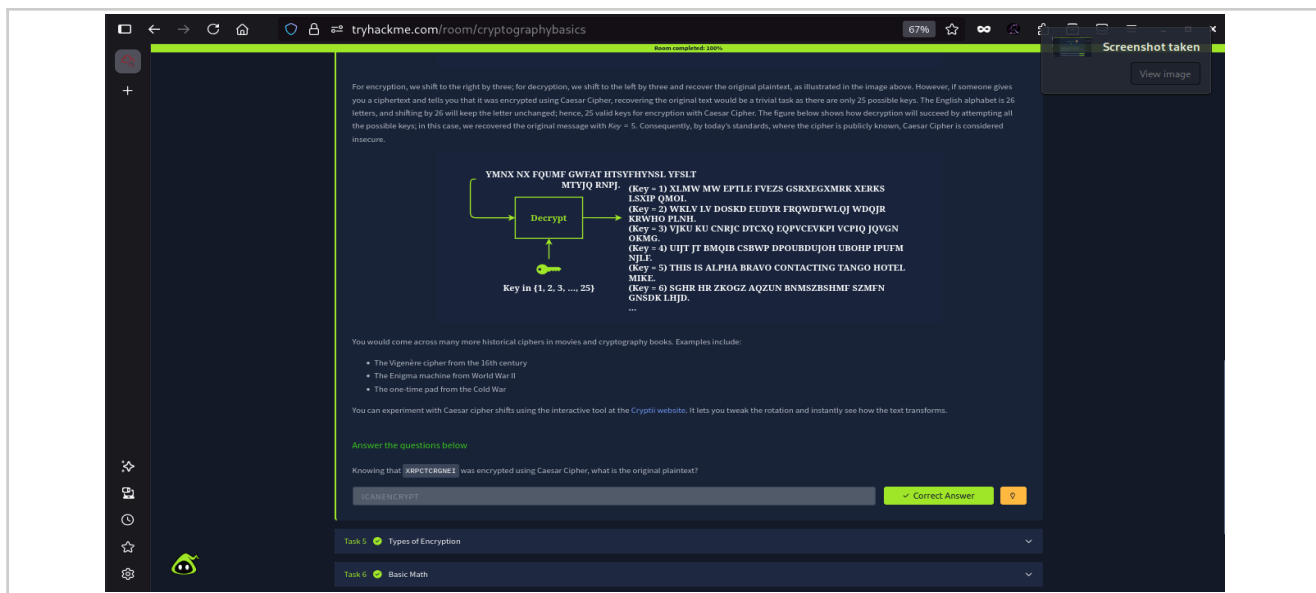
Task 4 — Historical Ciphers

This task walked through the Caesar cipher as the classic starting example — each letter in the plaintext gets shifted by a fixed number of positions in the alphabet, and the same shift in reverse decrypts it. It also mentioned a few other historical ciphers worth knowing about: the Vigenère cipher from the 16th century, the Enigma machine from World War II, and the one-time pad from the Cold War. There's also a link to the Cryptii website, which lets you experiment with Caesar shifts interactively and watch the text transform in real time.

The challenge question gave the ciphertext XRPCTCRGNEI and said it was encrypted with a Caesar cipher. I worked through the possible shifts (25 of them, since shifting by 26 just gets back to the original) until the output actually read as English, which turned out to be ICANENCRYPT. It's a good illustration of why the Caesar cipher is considered broken by modern standards — with only 25 possible shifts, brute-forcing it by hand takes a couple of minutes at most.

Q: Knowing that XRPCTCRGNEI was encrypted using Caesar Cipher, what is the original plaintext?

A: ICANENCRYPT.



Task 4 — Historical Ciphers

Task 5 — Types of Encryption

This task introduced Alice and Bob, the fictional pair used in basically every cryptography example to represent two parties trying to communicate securely, and drew the line between the two main types of encryption. Symmetric encryption uses the same key for both encryption and decryption, which means that key has to stay secret and never get shared with anyone outside the conversation. Asymmetric encryption uses two different keys instead — a public key that anyone can use to encrypt a message, and a private key, kept secret, that's the only thing that can decrypt it.

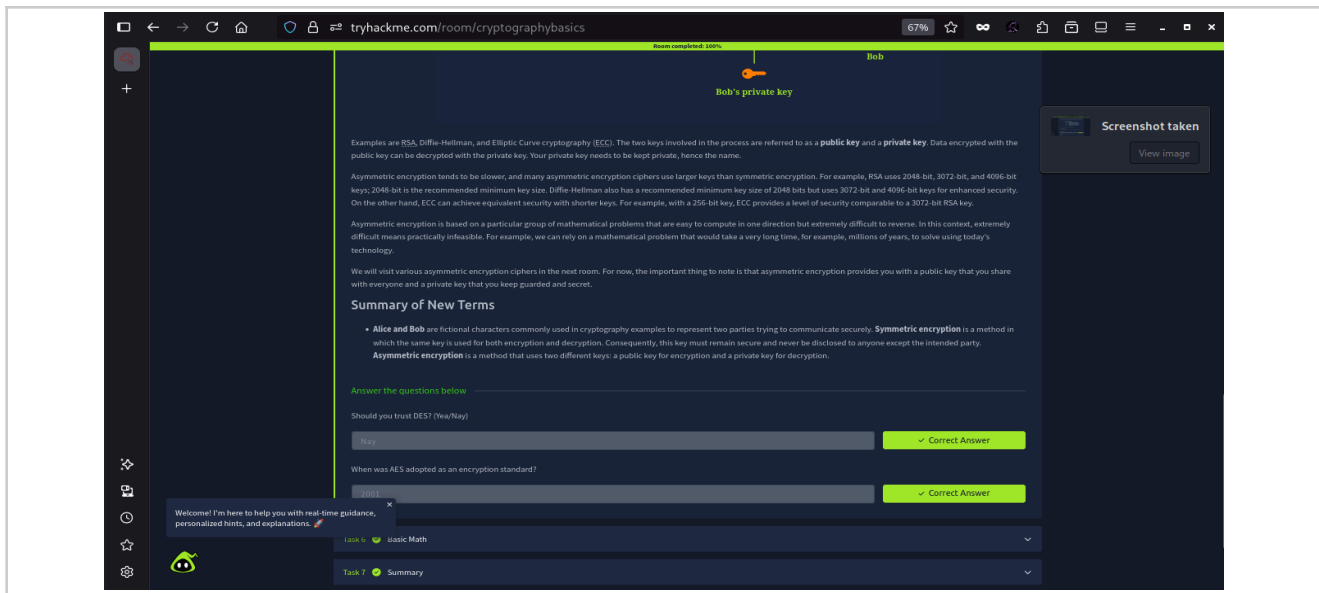
The task also touched on two symmetric algorithms specifically: DES and AES. DES uses a 56-bit key, which was considered secure decades ago but is now short enough to be brute-forced without much difficulty, which is why it's no longer trusted for anything serious. AES replaced it and was formally adopted as the encryption standard in 2001, and it's still what's used today.

Q: Should you trust DES? (Yea/Nay)

A: Nay — its 56-bit key is short enough to brute-force with modern hardware.

Q: When was AES adopted as an encryption standard?

A: 2001.



Task 5 — Types of Encryption

Task 6 — Basic Math

This task covered the two math operations that show up constantly once you get into how ciphers actually work under the hood: XOR and modulo.

XOR ($\oplus$) compares two binary numbers bit by bit and returns 1 where the bits differ and 0 where they match. Modulo (%) returns the remainder after dividing one number by another — so $23\%6$ is 5, because 23 divided by 6 is 3 with a remainder of 5. One thing worth remembering about modulo is that it isn't reversible: given only the result of $x\%5 = 4$, there are infinitely many values of x that could satisfy that, so you can't work backward to a single answer.

Q: What's $1001 \oplus 1010$?

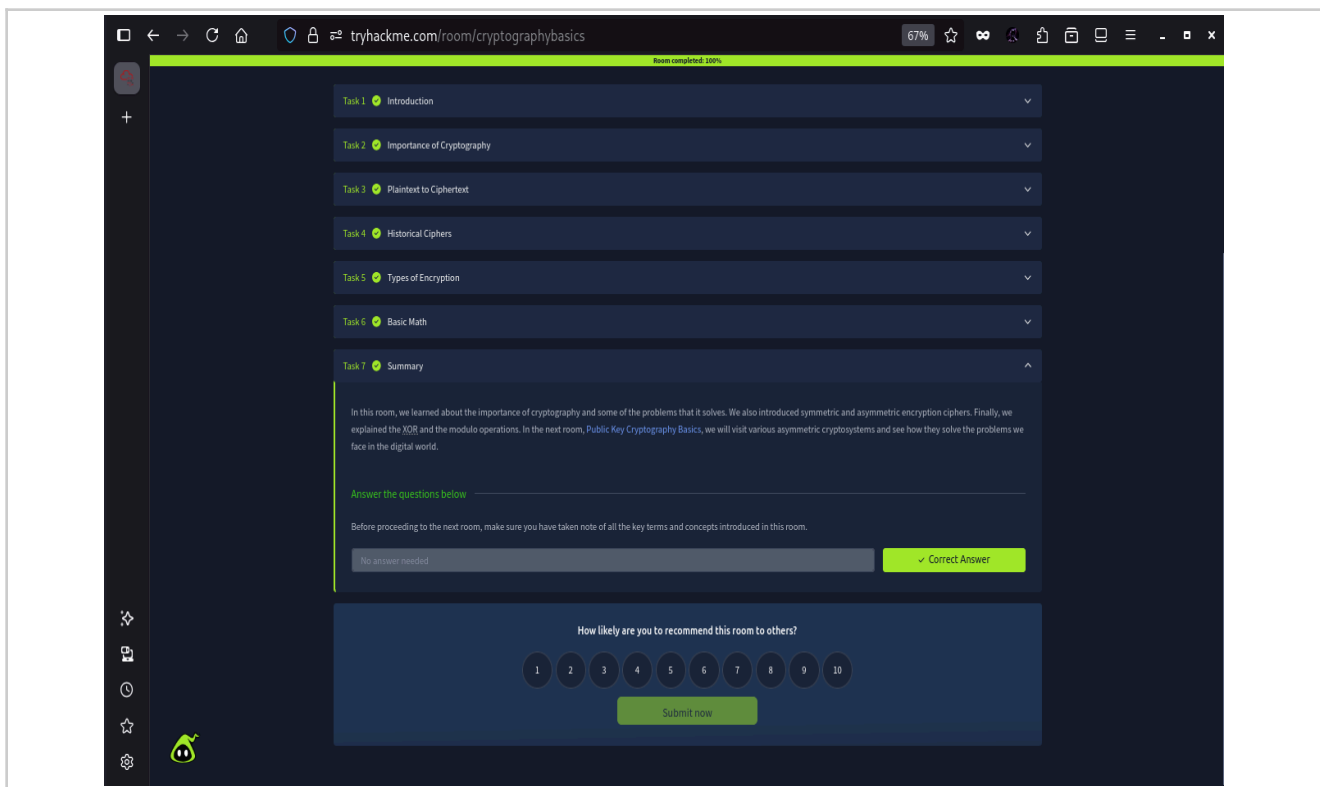
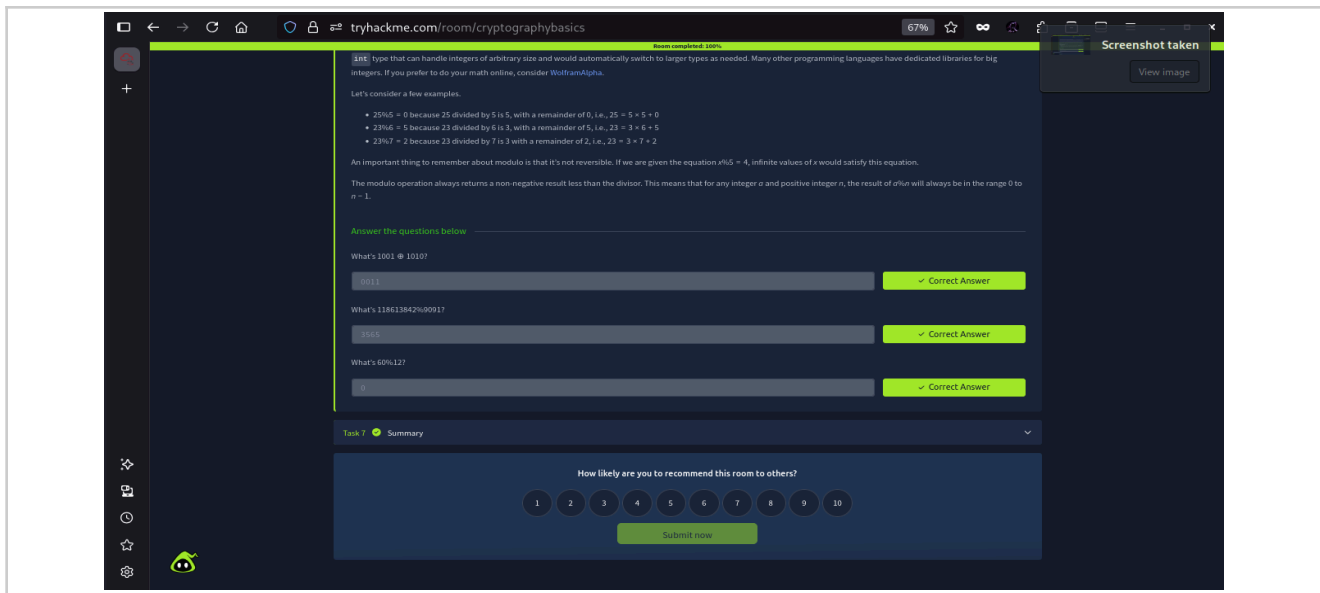
A: 0011 — comparing bit by bit: $1\oplus1=0$, $0\oplus0=0$, $0\oplus1=1$, $1\oplus0=1$.

Q: What's $118613842\%9091$?

A: 3565.

Q: What's $60\%12$?

A: 0 — 60 divides evenly by 12, so there's no remainder.



Task 7 — Summary, room completed at 100%

Public Key Cryptography Basics

This room goes deeper into asymmetric encryption, focusing on how algorithms like RSA actually work under the hood — generating a public/private key pair from large prime numbers, encrypting with the public key, and decrypting with the private key. It also covers Diffie-Hellman key exchange,

which lets two parties agree on a shared secret over an insecure channel without ever transmitting the secret itself, and touches on where this shows up in real life, like SSH authentication. The main takeaway is that asymmetric encryption solves the key-distribution problem that symmetric encryption has, at the cost of being slower and more computationally expensive.

Hashing Basics

This room covers hash functions — algorithms that take an input of any size and produce a fixed-length output (a hash/digest), where even a tiny change in the input produces a completely different hash. It explains why hashing is one-way (you can't reverse a hash back into the original input) and how that property makes it useful for two very different things: verifying file integrity (comparing hashes to confirm a file wasn't corrupted or tampered with) and storing passwords safely (storing a hash of the password instead of the password itself). It also introduces common hash algorithms like MD5, SHA-1, and SHA-256, and why the older ones (MD5, SHA-1) are now considered weak due to known collision vulnerabilities.

John the Ripper: The Basics

This room puts the previous two rooms into practice by introducing John the Ripper, a widely used password-cracking tool. It covers installing and running John, identifying the format of a given password hash, and the different cracking modes: wordlist mode (trying every password in a list like rockyou.txt), single-crack mode (using known info about the target to generate likely guesses), and brute-force/incremental mode (trying every possible combination). The room ties directly back into Hashing Basics — since passwords are usually stored as hashes rather than plaintext, cracking them means hashing a huge number of guesses and checking for a match rather than reversing the hash directly.

Conclusion

Across the four rooms, the module builds a complete picture of how encryption and hashing work together in practice: symmetric and asymmetric encryption protect data in transit and at rest, hashing protects data integrity and password storage, and tools like John the Ripper show why weak passwords and outdated hashing choices are still one of the easiest ways into a system. It's a solid base to build on before moving into more applied offensive or defensive security rooms.